KAVACH: defensible USP and Infosys gap research

Conclusion: KAVACH is an enterprise action-control component. Its strongest pitch is to combine argument-origin checks, staged or typed consequences, business facts and controlled release at the tool boundary. It can extend Infosys's existing platform controls. A new LLM, a generic guardrail wrapper, an approval queue, or “we block prompt injection” is not a defensible standalone USP.

Suggested pitch:

> KAVACH validates a proposed enterprise action against where its arguments came from, what it will change, and the business records that authorize it. Supported controlled actions run with a real safeguard and a verifiable decision-to-effect trail.

The execution qualification matters: proxy-only and hook-only deployments cannot automatically provide KAVACH's local holds or undo.

1. Scope and evidence quality

The two official Infosys repository heads were queried through the connected GitHub tool, and the review is pinned to:

Official repository	Inspected commit	Scope
Infosys/Infosys-Agentic-Foundry	`49fe687e68e60656fb75397d504739b11ddbe5ca`	Foundry, Topaz_AgentOS, tool governance, hooks, approval rules, SafeKernel, evidence, finance examples and the RAI integration
Infosys/Infosys-Responsible-AI-Toolkit	`5d65aefbff2b949bdfe374d9bb2fe2d83c9e00c4`	Moderation, privacy, explanation, benchmarking, injection-model implementation and module availability

The complete trees contain 2,300 and 4,210 entries respectively. Forty-one selected official source/document files were fetched and inspected. This is a targeted implementation review, not a claim to have executed every feature in either external platform. The local repository was reviewed across its core gateway, source/impact/policy engines, integrations, execution, audit, UI, operations and benchmark/test modules. Changes already present in application files were left untouched.

Evidence categories used throughout:

- **Source-confirmed:** a pinned implementation or official document supports the mechanism. Absence claims are limited to the reviewed path.
- **Fresh local test:** a test or pure-function probe ran in this workspace, using a copied repository or temporary source cache as appropriate.

- **Saved experiment:** checked records of another run; not independently reproduced here.
- **Inference:** product positioning or a plausible business motivation, explicitly identified.

Fresh checks: **521 local tests passed, 14 skipped, 0 failed**; **12/12 mechanism probes passed**. A seed-0 comparison using the actual fetched official gate functions was rerun on 282 synthetic cases. No live LLM, real-bank API, full AgentOS tenant deployment, or RAI classifier weights were exercised for this research. Detailed JSON artifacts are in this folder. Source manifests include commit/blob identities; no work dates or authoring timestamps are added to this report.

2. Clear up the categories first

Component	Job	What that means for KAVACH
LLM	Interpret a request and generate text or structured tool arguments	KAVACH consumes proposed actions; it does not train a replacement foundation model.
Agent runtime	Choose steps, invoke tools, manage memory, approvals and state	AgentOS/Foundry already provide this. KAVACH supplies additional checks at the invocation boundary.
Content moderation	Score/filter prompts and responses for injection, toxicity, PII or other content risks	RAI Toolkit already provides this. KAVACH adds authorization and consequence checks for the proposed operation.
Infrastructure sandbox / RBAC	Restrict paths, identities, commands, credentials and network reach	Necessary protection already present in AgentOS. KAVACH should reuse these boundaries.
Action validator	Decide whether this specific invocation is authorized and acceptable in this task	This is KAVACH's useful product scope.
Execution/safe guard owner	Actually hold, release, cancel or restore an effect	KAVACH can do this only for supported effects it controls or an integration explicitly delegates to it.

For client-defined tools, a model can return the invocation while application code executes it. That is the interception opportunity KAVACH uses; it is not a property of a “smarter model.” Anthropic's tool-use documentation describes this model/application split. Some providers also run server-side tools, so this is not a universal statement about every tool.

Concrete illustration: a finance employee may be allowed to use the payment API, yet a particular account number may come from an untrusted invoice email and differ from the vendor master. Tool permission and syntactically valid arguments do not establish transaction authorization. KAVACH's implemented R41 compares the intended account/amount with the records, and R18 separately handles release authority. Evidence: finance rules line 460, invoice regression line 56.

3. What Infosys already has

Agentic Foundry

Foundry provides agent templates, tools, MCP integration, response validators, tool validation, secret management, monitoring and human tool review. Its Tool Interrupt UI can show parameters and let a reviewer modify/approve them; Selective Tool Interrupt applies that to selected tools. Tool Validation includes side-effect and credential checks and recommends previews or dry runs. These are real overlaps. Evidence: Tool_Interrupt.md line 5, Tool_Validation.md line 54, MCP_Registry.md line 1.

Topaz AgentOS

Topaz_AgentOS is inside the official Foundry repository, not an unrelated repository with a similar AgentOS name. It includes multiple execution modes, SafeKernel, tenant scoping, memory, scheduling, durable approvals and hooks. The actual governance sequence runs pre-tool hooks, checks dangerous operations, and creates approval holds. It also attests the MCP connection configuration for held calls and refuses stale/unattestable configuration on resume. We must not claim that it has no automatic gate, no approval persistence, or no stale-approval protection. Evidence: README.md line 44, tool_governance.py line 223, agent.py line 1328.

Responsible AI Toolkit

The toolkit has prompt-injection and jailbreak checks, privacy detection/anonymization/encryption, toxicity and other moderation, explanation and benchmarking. Explanation includes response-oriented methods and token importance; it is not accurate to say Infosys has no explainability or uncertainty assessment. Its README marks particular Security and Red Teaming modules retired in the inspected release, but active moderation still implements prompt-injection checks. “Retired modules” does not mean all safety functionality disappeared. Evidence: RAI overview line 9, injection model line 64, LLM explanation line 17.

4. The strongest gaps, with precise boundaries

Gap A: collected evidence does not automatically constrain tool arguments

Existing capability: AgentOS collects source documents, reads, hashes and provenance/evidence metadata. It is wrong to claim it never records provenance. Its world-model module also has confidence-gated knowledge updates. Those capabilities are not the same as a security label attached to each proposed tool argument and used by the reviewed pre-tool gate.

Observed gap: the default gate reviewed here bases the decision on hooks, tool/skill names, and command/argument patterns. I did not find a built-in join in that path saying “this recipient came from this untrusted email, therefore this send must not run.” Custom hooks can supply such behavior; the framework is extensible.

KAVACH coverage: `register_output` records source trust/confidentiality; `label` traces values; R1/R11 constrain externally supplied control values, while R38/R40/R42 handle some missing provenance and user-only choices. This makes source information operational in the decision rather than merely available to a reviewer.

Limit: normal mode infers lineage through value matching and normalization. It is not complete tracking of arbitrary transformations by an LLM. Named sources and delegated tasks can require human confirmation instead of a hard denial. Typed planner mode has a stronger reference-flow design but its live generation remains incomplete.

Evidence: AgentOS evidence collector line 554, AgentOS decision path line 223, kavach/provenance.py line 233, kavach/policy.py line 251, kavach/planner.py line 80.

Gap B: a pattern hit does not measure the business effect

Existing capability: AgentOS's dangerous-call classifier recognizes configured tool names and regexes for commands, keywords and SQL. SafeKernel separately restricts execution. Foundry's onboarding guidance recommends safe tool design and possible dry-run support.

Observed gap: the reviewed built-in gate does not clone the business database and count changed rows/columns as part of every authorization decision. A broad SQL mutation regex can put a narrow update and an unbounded update into the same approval category. A provider may implement extra tool-specific controls; this is not a claim that every deployment lacks them.

KAVACH coverage: supported SQLite operations are staged on a copy, actual changes are collected, and sensitive record/column rules use the measured result. Other effects have typed models for recipients, data sensitivity, amounts and reversibility. The card states whether the result is sandboxed, static or predicted.

Limit: the demonstrated SQL database is local SQLite, and the enterprise data is seeded. A remote SQL/API call is not automatically staged against the real production state. A correct manifest and faithful enterprise adapter are prerequisites.

Evidence: agent.py line 813, Tool_Validation.md line 61, kavach/consequence.py line 188, kavach/changelog.py line 74, `official_source_probes.json`.

Gap C: policy must use authoritative facts even when the agent skips asking

Existing capability: AgentOS has a deterministic `evaluate_rules` engine as well as a prompt compiler. The evaluator supports thresholds, conditions and always/never approval rules. Saying “all business policy is only an LLM prompt” is too broad.

Observed gap: in the inspected `SkillAgent` path, `evaluate_rules` is called inside the agent-invoked `request_human_approval` tool, using the supplied context. The ordinary tool gate uses its separate pattern/hook policy. I did not find that generic gate automatically reading the vendor master/invoice amount for every payment call. An appropriate pre-tool hook or business API can fill this gap.

KAVACH coverage: the consequence/policy path fetches invoice/vendor facts from its environment; R18 uses the invoice's amount and role tiers; R41 refuses a mismatched intended account or amount; R35 blocks ordinary SQL from rewriting the protected records that those rules trust. This is more

specific than scanning an amount written by the agent.

Limit: the shipped facts are authoritative only for KAVACH's demo company. A genuine ERP/vendor master integration is needed before claiming the same fact fidelity in a client deployment. Sanctions and account-holder demonstrations are not certified real-world compliance services.

Evidence: approval_rules.py line 44, agent.py line 4046, tool_governance.py line 223, kavach/policy.py line 460, kavach/environment.py line 390, tests/test_gaps.py line 67.

Gap D: content moderation can miss the point where untrusted instructions enter

Existing capability: the RAI model can classify submitted text, including prompt injection. This is useful and reusable; KAVACH does not replace its broader checks.

Observed integration gap: the reviewed Foundry LiteLLM pre-call path selects `user`, `human` and `system` messages and extracts user-oriented prompt sections. It excludes role `tool`. The separate regex PromptGuard hook factory registers a scan of the user query at agent start. Consequently, an injected email returned as a tool result is not submitted by those specific paths. Turning them on does not change their role-selection code. A different integration can scan that output.

KAVACH coverage: untrusted tool outputs are registered for later argument tracing, so a malicious destination can be stopped even if the text contains no recognizable jailbreak phrase. The optional CPU injection probe can provide additional escalation signals, but it is disabled by default. There is no shipped claim here that KAVACH actively calls the Infosys RAI service.

Limit: integration visibility matters. The hook runner can redact and truncate output; KAVACH cannot trace text it never receives. Conservatively escalating unfamiliar destinations helps, but is not proof of complete security.

Evidence: custom_guardrail.py line 331, prompt_guard.py line 273, kavach/gateway.py line 207, kavach/gateway.py line 64, hook_runner.py line 1013.

Gap E: asking before execution does not provide cancellation after approval

Existing capability: AgentOS can hold a proposed call for review and durably resume it. Foundry has tool-code version rollback. These are valuable, but neither establishes a generic business-effect reversal contract for every approved tool.

Observed gap: I did not find a common email-queue/payment-hold/row-restoration mechanism in the inspected governance path. An enterprise tool or API may supply its own cancellation, drafts or compensation; the platform can integrate it.

KAVACH coverage: when it owns execution, supported SQL/files have snapshots/change logs; email waits in an outbox; payments can wait in a hold and be rechecked; SAP demo entries are parked. Cancellation happens before the irreversible effect. Recovery persists waiting state; uncertain delivery is recorded rather than blindly repeated.

Limit: there is no recall of delivered mail, no general real-bank reversal and no universal “undo.” A refund can run without a reversible safeguard. Proxy/hook modes that leave effects to other executors do not gain these local guarantees. No general exactly-once guarantee is claimed for

arbitrary remote effects.

Evidence: kavach/gateway.py line 757, kavach/environment.py line 518, kavach/windows.py line 17, tests/test_email_window.py line 112, tests/test_external_execution.py line 17.

Gap F: ordinary evidence storage is not an independently verifiable decision chain

Existing capability: AgentOS has audit logging, evidence persistence, content hashes and configurable JSONL/OTel sinks. Its document collector can HMAC-sign download URLs. Those should not be dismissed as “no audit” or “no hashes.”

Observed difference: the inspected audit/evidence path does not show the same per-decision linked hash chain plus signed chain-head checkpoint contract used by KAVACH. Hashing a source document or signing a download URL is a different control.

KAVACH coverage: receipts record rule, source, decision and effect references; the chain uses SHA-256/HMAC and supports Ed25519 verification. A separate signed checkpoint detects a cut-short tail. A configured durable store must accept a decision/approval/early-release record before the corresponding action proceeds.

Limit: proper key/storage management is required. An attacker with all trusted keys and stores is outside a simplistic “immutable” claim. External anchoring is optional and was not exercised. Human-readable evidence does not certify legal compliance.

Evidence: audit_log.py line 30, evidence_store.py line 65, document_evidence.py line 524, kavach/receipts.py line 44, tests/test_checkpoint.py line 37, tests/test_receipt_store_down.py line 18.

5. What is overlap, not a USP

Candidate claim	Assessment
“We have pre-tool hooks / an MCP proxy.”	Established integration pattern; AgentOS already has hooks and governance. Useful packaging, not unique science.
“We can ask a human.”	Already a major Foundry/AgentOS feature. Improve the evidence and selection of exceptions.
“We bind approvals / detect changed MCP configuration.”	Overlap. AgentOS already stores held calls and checks MCP configuration on resume. KAVACH's digest and tool-definition pins are specific complementary mechanisms.
“We have RBAC, tenant support, secrets and telemetry.”	Baseline infrastructure. Reuse the platform; do not sell its absence.
“We detect PII or injection.”	RAI Toolkit already does this, and covers modalities KAVACH does not.
“We use AI to explain decisions.”	RAI already has explanation features. KAVACH's useful difference is the action-specific evidence behind an explanation.

Candidate claim	Assessment
“We learn from feedback.”	AgentOS also has learning/evaluation workflows; KAVACH's on-demand gated retraining is a component, not a proven moat.
“We are fail-closed everywhere.”	Too broad. AgentOS has fail-closed controls and configurable hook policies; KAVACH falls back on deterministic checks when advisory models fail. Not every optional signal's failure causes denial.
“42 rules make us novel.”	Rule count is an implementation inventory. Generalization, fact fidelity and demonstrated coverage matter more.

The RAI integration docs describe a fail-open approach, while the reviewed pre-call code raises a blocking error on some API failures and skips when pre-call moderation/configuration is absent. This is path/configuration-dependent. Do not generalize one default into “the whole Infosys stack fails open.” Evidence: RAI_Guardrails.md line 248, custom_guardrail.py line 408, hook_runner.py line 520.

6. Comparison with the wider agent-security field

Existing approach	What is already established	KAVACH's defensible position
LLM client-tool calling	Model proposes; application executes	Insert a verified business-action check before application-side execution.
Meta LlamaFirewall	Layered injection/intent/code scanners	KAVACH emphasizes origin/effect/business facts and controlled delivery; it has not demonstrated an overall head-to-head win.
CaMeL	Control/data separation, capabilities and tool-call security policy	Provenance is not a new invention. KAVACH's normal matching is weaker than CaMeL's constructed data-flow contract; its planner is exploratory.
AWS AgentCore Policy	Deterministic gateway authorization; temporal/session conditions and computed signals	Policy engines and cumulative budgets are not unique. KAVACH supplies concrete effect/record evidence and local safeguard workflows in its prototype.

Primary sources: LlamaFirewall paper, official LlamaFirewall README, CaMeL paper, AWS AgentCore Policy.

The honest novelty claim is an implemented combination and an Infosys-oriented integration, not that no other system has provenance, policy, agent interception or safeguards. No patentability or exhaustive market-uniqueness conclusion is supported by this review.

7. Fresh comparison: useful engineering evidence, not a platform leaderboard

The repository harness loads reviewed functions from the pinned official source: PromptGuard regex scanning, AgentOS pattern classification, and the proxy's extraction path. It supplies a perfect detector only for text the RAI selection path would submit. No RAI model weights run. No full SafeKernel, custom tenant hooks, production API safeguards or human review outcomes are simulated comprehensively.

The fresh seed-0 run used **282 test cases: 156 unsafe and 126 safe**, sampled from the project's 940-case, 47-family synthetic suite. The team wrote both rules and cases. Rates below are conditional on that harness.

Harness configuration	Unsafe auto-allowed	Injection success	Safe auto-completed	Escalated
AgentOS default gate, autonomous	96.2%	100.0%	100.0%	0.0%
AgentOS hardened gate	3.9%	0.0%	14.3%	89.4%
Harness interrupt-every-change baseline	3.9%	0.0%	9.5%	93.6%
KAVACH full	0.0%	0.0%	95.2%	26.6%

The exact contribution supported here: the configured coarse baseline can stop the injection families by escalating broadly, whereas KAVACH's purpose-built fact/source/effect checks automate more labelled-safe cases on this suite. It does **not** establish that deployed Infosys products let 96.2% of attacks through.

The “interrupt every change” baseline still allows 3.9% of unsafe cases because the remaining unsafe family is `attack:secret_read`: its tool is classified as a read, and the baseline interrupts writes rather than sensitive reads. It is not evidence that approvals of state changes were bypassed. A fairer evaluation must specify whether it protects sensitive reads too and account for the human's decision quality and burden. Evidence: bench/run.py line 77, bench/suite.py line 173. The baseline name alone does not describe protection of sensitive reads.

These are decision-verdict rates: the benchmark checks proposed calls, and an escalation counts as not automatically allowed. It does not measure actual incident frequency, human approval accuracy or production task completion.

The fresh artifact is `infosys_seed0_recheck.json`. Historical multiseed figures remain separate: KAVACH's recorded five-seed mean is 95.2% safe automation and 26.9% escalation; this fresh seed-0 run is 26.6% escalation. The smaller saved comparison file's other run reports 25.5%; these should not be silently mixed.

8. The AI/model story must be restrained

The `layers.json` ablation reports no verdict changes when learned risk is forced to zero. Rules and the intent gate determine those synthetic outcomes. The model is a supporting signal; the suite does not demonstrate that it causes the safety improvement. Fixed and reversibility-conditioned thresholds tie on aggregate metrics. A strong calibration result on a narrow family distribution does not prove broad robustness.

The default server mode runs the Qualcomm judge on escalations, for reviewer advice. A separately saved 24-case inline semantic test reports 7/12 misreadings caught versus 1/12 with the heuristic, while safe completion is 11/12 in both. Those are author-written examples; five unsafe misreadings remain uncaught. Optional explanation rewrites have rejection checks; templates remain the default. Evidence: bench/results/LAYERS.md line 23, kavach/gateway.py line 53, bench/results/SEMANTIC.md line 5.

Useful pitch: “AI can interpret uncertain intent and explain the evidence; deterministic policy retains authority.” Unsupported pitch: “Our unique calibrated AI guarantees secure autonomy.”

9. The live evidence is mixed and must stay visible

The saved paired AgentDojo banking summary has 65/128 attacker successes without KAVACH and 0/128 with it, with benign completion 8/16 in both. The paired Slack summary has 48/103 versus 11/103 attacker successes, but benign completion falls from 9/21 to 6/21 and user-task completion under attack falls from 36/103 to 21/103.

These are saved partial runs on one model/attack setup; raw live-task logs are absent from this checkout, so the external experiments were not independently reproduced here. They do not prove universal safety or preserved utility. Escalations count as blocks because no reviewer participates.

The offline replay has per-task records, but it is a different experiment. Slack's official attempted-call score can count a blocked attempt as success; executed-only scoring differs. Do not substitute executed-only zero into the live official metric. The saved Claude comparison also shows the model refusing malicious requests without KAVACH: that refusal belongs to the model.

Evidence: bench/results/COMPARISON.md line 11, bench/results/AGENTDOJO_REPLAY.md line 17, bench/results/CLAUDE_WITH_WITHOUT.md line 3.

10. Why Infosys might want this built

Inference from the official product scope and the provided challenge description: enterprises need agents that can operate business tools under concrete operational constraints. Foundry supplies agents and tools; AgentOS supplies execution/governance; RAI supplies content/model risk controls. A reusable action validator can add transaction-specific evidence and policy without making every application team write a separate collection of checks.

That is compatible with the stated ET-03 themes of safety, authorization, explanation and reversibility. I did not find an authoritative public ET-03 requirements page in this search. The brief's wording here comes from the project-provided materials. I cannot claim to know Infosys's private judging intent, roadmap or product gaps beyond what the reviewed public paths establish.

Likely business value to test:

1. Fewer inappropriate actions reach enterprise systems.
2. Fewer routine cases need manual escalation, at acceptable false-negative and false-positive rates.
3. Reviewers see the relevant record, data origin and proposed effect before authorizing a call.
4. Operators can stop supported waiting effects and investigate uncertain outcomes.
5. Integration teams reuse one effect/provenance contract across workflows.

No measured ROI, reduced incident cost or customer adoption is established here. Demo `cost_per_exception` metadata is not a validated savings estimate.

11. How it should fit into the Infosys stack

Recommended division of responsibility:

```
User request + authenticated identity
                |
Agentic Foundry / Topaz AgentOS
  reasoning, planning, routing, memory, scheduling
                |
PreToolUse -> KAVACH
  argument origin + staged/typed effect + business facts
  deterministic denial / approval floor + explanation
                |
AgentOS approval operations + SafeKernel / tool host
                |
Business tool / enterprise API
                |
PostToolUse + full-output registration + actual outcome
                |
Evidence / receipt / telemetry
```

RAI content checks can supply additional signals on appropriately selected user inputs, untrusted outputs and outgoing content. The architecture above is a proposed combined deployment; KAVACH does not currently integrate the RAI API as a tested first-class backend.

In a hook integration, AgentOS keeps the approval queue, authenticated caller roles and executor. KAVACH returns denial/escalation evidence. For local holds or snapshots, either route those particular effects through KAVACH-owned connectors or implement an explicit cancellable/stage-and-commit adapter contract. Do not execute the same action once in KAVACH and again in AgentOS.

12. Concrete integration limitations uncovered

Limitation	Evidence	Implication
Hook tool outputs are redacted and truncated to 4,096 characters; user-query context is also bounded	AgentOS `HookRunner._build_env`	Missing content can weaken lineage precision. Full-output registration or explicitly conservative handling is needed. This is not a demonstrated bypass.
KAVACH hook reads `KAVACH_TOKEN`, but the reviewed external-hook environment allowlist does not propagate it	AgentOS hook environment + local `agentos_hook.py`	The example is not a complete authenticated production bridge. Add an approved credential-resolution path and test it; do not assume ambient variables arrive.
Context recording and the short decision cache are local-file based	Local `record`, `check`, `_session`	Need explicit tenant/user/task scoping, locking/retention and per-invocation decision identity before broad deployment. Shared-directory use deserves a separate security review.
AgentOS owns approval resumption; KAVACH hook is not informed through a completed approval callback protocol	Adapter + saved governance report	Do not claim end-to-end KAVACH approval binding or automatic local undo for this path.
Finance hook maps an invoice ID to KAVACH's records	`to_kavach`, `ap_invoice_facts`	The demonstration is not a real ERP fact adapter. Synchronization, trust and stale-record handling must be designed.
Local KAVACH approval validates the stored digest, not every policy/pin again	`resolve_hitl`	Add policy/tool/state revalidation as required for production; do not overstate it now.
Check-only can predict a safeguard before an effect is actually armed	API `safeguard_armed`	Show actual execution status, not only a safeguard-capable verdict.

Exact sources: hook_runner.py line 931, hook_runner.py line 1013, integrations/agentos_hook.py line 91, integrations/agentos_hook.py line 104, integrations/agentos_hook.py line 118, kavach/gateway.py line 918.

Correction to the earlier deck audit: `/api/v1/check` is not strictly incapable of execution. `CheckIn.execute` defaults to `False`, but `execute=True` calls `gw.execute`. The accurate statement is "decision-only by default; optional owned execution for supported local tools." This was verified by source/AST probes. Evidence: kavach/server.py line 198, kavach/server.py line 335.

13. Strongest demonstration of the USP

Use invoice INV-101 as a business story:

1. The user asks to prepare payment using approved vendor details.
2. The inbox contains conflicting bank-account instructions.

3. KAVACH reads the invoice/vendor facts; substituted account TEST-ACCT-999 is refused, rather than accepted because it is a valid-looking string.
4. The approved account TEST-ACCT-001 for INR 8,000 still requires the appropriate human release authority.
5. On owned execution the payment enters a hold. A reviewer can cancel while it is waiting.
6. Show the sandbox bank ledger and correlated receipt, rather than relying on the agent's claim of success.

Then contrast a legitimate user-specified email with an injected external destination and a narrow CRM update with a broad one. These exercise authorization, source, impact and controlled release. The real bank is simulated. Label chosen/scripted calls honestly. Evidence: tests/test_invoice_demo.py line 65, tests/test_trace.py line 22.

14. What to say to judges

Short pitch:

> Infosys already has agent orchestration, content moderation and approval workflows. KAVACH adds a reusable business-action check: it traces the proposed arguments, measures or predicts the effect, applies policy to trusted records, and returns evidence for a safe execution or a human decision. Supported actions under its control have real cancellation or undo mechanisms.

If asked "why not just use a better LLM?" Model quality helps but does not establish authorization for a particular bank account, recipient or set of records. We enforce those facts outside the model. We do not claim every model will fail the same injection.

If asked "why not just enable HITL?" HITL can already prevent the action if reviewers examine everything. We add concrete source/effect evidence and try to limit unnecessary review. The current synthetic comparison motivates that design; reviewer fatigue or economic savings were not measured in a user study.

If asked "isn't this just policy-as-code?" The policy evaluator is one part. Much of the work is producing the facts it evaluates—origin labels, row/column diffs, invoice/vendor values—and correctly binding the decision to a supported executor. Existing policy engines can consume similar facts, so this is an implementation/integration value claim, not exclusive ownership of the idea.

If asked "can Infosys build this using its hooks?" Yes. Its hooks are precisely the extension seam. KAVACH packages a domain-aware implementation of checks that are not automatically supplied by those generic hooks. That is complementary product value.

15. What not to say

Avoid	Use instead
“Infosys has no guardrails.”	“The reviewed built-in paths do not combine these source/effect/business-fact checks by default.”
“No other product does runtime policy or provenance.”	“Our prototype combines established controls for this enterprise workflow.”
“Our model is why attacks stop.”	“Rules and intent checks carry the saved synthetic outcomes; ML supports review.”
“Zero attack success.”	“Zero on the specified banking subset / synthetic split; Slack has residual successes and utility loss.”
“Fully verified in AgentOS.”	“Adapter and saved governance-code checks exist; an authenticated LLM-driven production tenant flow remains unproven.”
“All actions are reversible.”	“Supported owned-execution effects can have snapshots, holds or parking, with explicit boundaries.”
“96.2% of Infosys is unsafe.”	“One limited default-gate harness allowed 96.2% of this team's unsafe synthetic cases.”
“Production ready.”	“Demonstrated workflows with named integration and operational work remaining.”

16. Recommended next work, in priority order

1. **Build one real business adapter.** Bind a vendor/invoice source of truth to policy and execution, preserve record versions, and revalidate before release. Keep tests on synthetic records while verifying the real connector contract.
2. **Complete authenticated AgentOS integration.** Propagate scoped identity safely; register outputs without silent truncation; use tenant/task-scoped storage; record approvals and actual effects through supported callbacks.
3. **Make the decision-to-execution contract explicit.** Include action digest, tool/config hash, policy version, record version and one-use/expiry semantics. Recheck relevant state at execution. This closes a larger proof gap than adding another optional model.
4. **Improve independent evaluation.** Equal tool coverage, identical configurations, held-out families, multiple models/attacks, preserved raw traces, official scoring and separate safety/utility/escalation measures. Rerun RAI classifiers if making a classifier-quality claim.
5. **Validate human review.** Compare cards and routing with actual reviewers. Measure wrong approvals, missed attacks and unnecessary interrupts; avoid inventing a fatigue benefit.
6. **Finish operational readiness.** CI, load tests, backup/restore, alert routing, deployment/key-management hardening and a precise concurrency design.

These are recommendations inferred from the implementation and evidence gaps, not capabilities already delivered.

17. Source and artifact index

Local research artifacts:

- `infosys_seed0_recheck.json`: fresh, scoped comparison metrics and family verdicts.
- `official_source_probes.json`: twelve narrowly defined function/AST checks.
- `repository_tests.json`: 521 passing tests, fourteen skips and their reasons.
- `source_manifest.json`: pinned official repository files, blob IDs, fetch URLs and source hashes.
- `repository_map.json`: end-to-end local component inventory and review boundaries.

The existing `docs/INFOSYS_GAP_ANALYSIS.md` contains conflicting old percentages and some overbroad language. This report supersedes it for USP/research claims; it was not overwritten because it already had workspace edits. Likewise, the earlier deck/reference material should use the more precise REST, approval-evaluator and evidence/provenance language recorded here before making those particular claims.

Pinned official files reviewed:

- Infosys/Infosys-Agentic-Foundry: docs/RAI_Guardrails.md
- Infosys/Infosys-Agentic-Foundry: README.md
- Infosys/Infosys-Agentic-Foundry: docs/Tool_Interrupt.md
- Infosys/Infosys-Agentic-Foundry: docs/Tool_Validation.md
- Infosys/Infosys-Agentic-Foundry: docs/MCP_Registry.md
- Infosys/Infosys-Agentic-Foundry: Infosys-Agentic-Foundry-Backend/src/models/guardrail_aware_llm.py
- Infosys/Infosys-Agentic-Foundry: IAF-Litellm-Server/custom_guardrail.py
- Infosys/Infosys-Agentic-Foundry: Infosys-Agentic-Foundry-Backend/src/agentos/prompt_guard.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/docs/HOOK_BASED_HITL_FLOW.md
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/docs/HITL_DESIGN_GUIDE.md
- Infosys/Infosys-Agentic-Foundry: Infosys-Agentic-Foundry-Backend/src/utils/guardrail_helpers.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/README.md
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Skills/iaf_agent_skills/approval_rules.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Skills/iaf_agent_skills/hook_runner.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Skills/iaf_agent_skills/agent.py

- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Skills/iaf_agent_skills/tool_governance.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/agent_os/audit_log.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/agent_os/egress_ssrf.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/agent_os/safe_path.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/agent_os/kernel.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Skills/iaf_agent_skills/world_model.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Skills/iaf_agent_skills/entity_validation.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Skills/iaf_agent_skills/approval_store.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Skills/iaf_agent_skills/document_evidence.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/enterprise_root/tenants/company/skills/payment_processing/SKILL.md
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/enterprise_root/tenants/company/skills/trace_value_router/SKILL.md
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/enterprise_root/tenants/company/shared/contexts/judgment/fa_judgment.md
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/enterprise_root/tenants/company/skills/invoice_processing_ap/SKILL.md
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Studio/shared/evidence_store.py
- Infosys/Infosys-Agentic-Foundry: Topaz_AgentOS/IAF-Agent-Studio/shared/audit_evidence_sinks.py
- Infosys/Infosys-Responsible-AI-Toolkit: README.md
- Infosys/Infosys-Responsible-AI-Toolkit: responsible-ai-moderationlayer/README.md
- Infosys/Infosys-Responsible-AI-Toolkit: responsible-ai-moderationmodel/README.md
- Infosys/Infosys-Responsible-AI-Toolkit: responsible-ai-privacy/README.md
- Infosys/Infosys-Responsible-AI-Toolkit: responsible-ai-moderationmodel/src/service/injectionModel.py
- Infosys/Infosys-Responsible-AI-Toolkit: responsible-ai-moderationmodel/src/routing/injectionRouter.py
- Infosys/Infosys-Responsible-AI-Toolkit: responsible-ai-llm-explain/README.md
- Infosys/Infosys-Responsible-AI-Toolkit: responsible-ai-llm-benchmarking/README.md
- Infosys/Infosys-Responsible-AI-Toolkit: github-pages/content/Privacy.html
- Infosys/Infosys-Responsible-AI-Toolkit: github-pages/content/FM-Moderation_Features.html
- Infosys/Infosys-Responsible-AI-Toolkit: responsible-ai-moderationlayer/src/router/router.py